

Fiche 315 — Paquets et espaces de noms :

`library`, `::`, `:::`

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 13 « Packages » (§13.1 paquets standard, §13.2 paquets contribués et CRAN, §13.3 espaces de noms)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §3.5.4 « Search path » ; <i>Writing R Extensions</i> 4.6.1, chapitre 1 « Creating R packages » (renvoi explicite du §13)
Difficulté	Fondamental — court, mais il conditionne tout usage réel de R
Temps d'étude estimé	45 min
Prérequis	Fiches 306 et 310 (chemin de recherche, environnements)
Concepts clés	paquet, chargement, efficacité et protection contre les collisions , <code>library()</code> , <code>install.packages()</code> , <code>update.packages()</code> , <code>search()</code> , <code>loadedNamespaces()</code> , paquets standard , recommandés et contribués , CRAN, Bioconductor, espace de noms et ses trois rôles , <code>::</code> , <code>:::</code> , <code>getAnywhere()</code> , chargement automatique
À retenir en priorité	Les trois rôles d'un espace de noms · <code>::</code> n'atteint que ce qui est exporté · <code>search()</code> ne montre pas tout · un paquet chargé automatiquement n'entre pas dans le chemin de recherche.

Vue d'ensemble

LE PRINCIPE	« ALL R functions and datasets are STORED IN PACKAGES. ONLY WHEN A PACKAGE IS LOADED are its contents available. »	
DEUX RAISONS	EFFICACITE	une liste complete prendrait plus de memoire et serait plus longue a parcourir
	DEVELOPPEURS	« PROTECTED FROM NAME CLASHES with other code »
LES COMMANDES	<code>library()</code>	ce qui est INSTALLE
	<code>library(boot)</code>	CHARGER un paquet
	<code>install.packages()</code>	installer

<code>update.packages()</code>	mettre à jour
<code>search()</code>	ce qui est CHARGÉ ET SUR LE CHEMIN
<code>loadedNamespaces()</code>	ce qui est chargé SANS y être

TROIS CATEGORIES

STANDARD (base)	« part of the R source code » -- toujours disponibles
RECOMMANDES	livres avec toute distribution binaire
CONTRIBUES	des milliers, sur CRAN et Bioconductor

UN ESPACE DE NOMS FAIT TROIS CHOSES

1. CACHER les fonctions et données à usage interne
2. PROTÉGER contre un nom choisi par l'utilisateur ou un autre paquet
3. CONCEVOIR un objet d'un paquet particulier

DEUX OPÉRATEURS	<code>base::t</code>	ce qui est EXPORTÉ
	<code>pkg::f</code>	y compris ce qui est CACHÉ
	<code>getAnywhere()</code>	cherche dans PLUSIEURS paquets

!! « When packages with namespaces are LOADED AUTOMATICALLY
they are NOT ADDED TO THE SEARCH LIST. »

Le problème posé. « *All R functions and datasets are stored in packages. Only when a package is loaded are its contents available.* » (§13)

INTUITION — LES DEUX RAISONS, ET ELLES NE SE RESSEMBLENT PAS.

« ***This is done both for efficiency*** (the full list would take **more memory** and would take **longer to search** than a subset), ***and to aid package developers, who are protected from name clashes with other code.*** »

La première est **une question de coût** : ne charger que ce qui sert. La seconde est **une question de correction** : garantir qu'un paquet continue de fonctionner quoi que l'utilisateur définisse par ailleurs. C'est cette seconde raison qui donne naissance aux **espaces de noms** (concept 3), et elle répond exactement au problème de masquage rencontré en fiche 306.

« The process of developing packages is described in Section "Creating R packages" in **Writing R Extensions**. **Here, we will describe them from a user's point of view.** » — la production d'un paquet est l'objet de la fiche 319.

● Concept 1 — Les commandes de l'utilisateur

Commande (§13)	Ce qu'elle fait
<code>library()</code> sans argument	« To see which packages are installed at your site »
<code>library(boot)</code>	« To load a particular package »
<code>install.packages()</code> · <code>update.packages()</code>	« Users connected to the Internet can use ... to install and update packages » — « available through the Packages menu in the Windows and macOS GUIs »
<code>search()</code>	« To see which packages are currently loaded ... to display the search list »
<code>loadedNamespaces()</code>	« Some packages may be loaded but not available on the search list ... these will be included in the list given by »
<code>help.start()</code>	« To see a list of all available help topics in an installed package ... then navigate to the package listing in the Reference section »

La distinction entre `search()` et `loadedNamespaces()` est le point pratique du chapitre. Un paquet peut être **chargé** — donc actif, ses fonctions utilisables par `::` — **sans figurer dans le chemin de recherche**. `search()` seul donne donc une vue **incomplète** de ce qui est en mémoire. La raison est donnée au §13.3 : « **When packages with namespaces are loaded automatically they are not added to the search list.** »

Le paquet cité en exemple est `boot`, « containing functions from **Davison & Hinkley (1997)** » — l'ouvrage sur le bootstrap déjà présent dans les références du manuel.

● Concept 2 — Les trois catégories de paquets

DÉFINITION (§13.1).

« **The standard (or base) packages are considered part of the R source code. They contain the basic functions that allow R to work, and the datasets and standard statistical and graphical functions that are described in this manual. They should be automatically available in any R installation.** »

DÉFINITION (§13.2).

« *There are thousands of contributed packages for R, written by many different authors. Some of these packages **implement specialized statistical methods**, others **give access to data or hardware**, and others are **designed to complement textbooks**.* »

« *Some (the recommended packages) are distributed with every binary distribution of R. Most are available for download from CRAN ... and other repositories such as Bioconductor.* »

« *The R FAQ contains a list of CRAN packages **current at the time of release**, but **the collection of available packages changes very frequently**.* »

Catégorie	Statut	Disponibilité
Standard (<i>base</i>)	« <i>part of the R source code</i> »	toujours
Recommandés	distribués avec toute distribution binaire	installés , à charger
Contribués	des milliers , auteurs divers	à installer depuis CRAN, Bioconductor...

La fiche 300 avait donné le chiffre : « *There are **about 25 packages** supplied with R (called "standard" and "recommended" packages)* » (§1.3). Ce sont les deux premières lignes du tableau réunies.

● Concept 3 — Les espaces de noms

DÉFINITION (§13.3).

« *Packages have namespaces, which do three things :*

1. « *they allow the package writer to hide functions and data that are meant only for internal use* » ;
2. « *they prevent functions from breaking when a user (or other package writer) picks a name that clashes with one in the package* » ;
3. « *they provide a way to refer to an object within a particular package.* »

L'exemple du cours (§13.3). *« For example, `t()` is the transpose function in R, but users might define their own function named `t`. Namespaces prevent the user's definition from taking precedence, and breaking every function that tries to transpose a matrix. »*

Voilà le problème de la fiche 306 résolu. Le chemin de recherche prend la **première correspondance**, donc un `t` défini dans l'espace de travail **masque** celui de `base` — pour l'utilisateur. Mais **pas pour le paquet**, dont le code trouve son `t` avant tout le reste.

COMPLÉMENT — LE MÉCANISME EXACT (R LANGUAGE DEFINITION §3.5.4).

« *Packages which have a namespace have a different search path. When a search for an R object is started from an object in such a package, the package itself is searched first, then its imports, then the base namespace and finally the global environment and the rest of the regular search path. The effect is that references to other objects in the same package will be resolved to the package, and objects cannot be masked by objects of the same name in the global environment or in other packages.* »

le paquet → ses imports → `base` → `.GlobalEnv` → le reste du chemin

Le paquet passe avant tout. C'est ce qui fait de son fonctionnement une propriété **de son code**, et non de l'état de la session.

3.1 Les deux opérateurs

DÉFINITION (§13.3).

« *There are two operators that work with namespaces.* »

- `::` — « *The double-colon operator `::` selects definitions from a particular namespace. In the example above, the transpose function will always be available as `base::t`, because it is defined in the base package. Only functions that are exported from the package can be retrieved in this way.* »
- `:::` — « *The triple-colon operator `:::` may be seen in a few places in R code : it acts like the double-colon operator but also allows access to hidden objects. Users are more likely to use the `getAnywhere()` function, which searches multiple packages.* »

Écriture	Portée
<code>paquet::objet</code>	uniquement ce qui est exporté
<code>paquet:::objet</code>	y compris ce qui est caché
<code>getAnywhere("objet")</code>	« <i>searches multiple packages</i> » — pour l'utilisateur

EN CLAIR — QUAND UTILISER QUOI.

- `::` est la **bonne écriture** dès qu'un nom peut être ambigu : `stats::filter` , `base::t` . Elle documente d'où vient la fonction et **survit à tout masquage**.
- `:::` franchit une frontière que l'auteur du paquet a **volontairement posée**. Le cours en parle au passé — « *may be seen in a few places in R code* » — et **oriente aussitôt vers** `getAnywhere()` . Un objet caché n'est pas dans le contrat public : il peut changer de nom ou disparaître sans préavis.

C'est la même distinction qu'en fiche 311, où `getS3method("generic", "class")` est « *la voie officielle* » pour lire une méthode non exportée.

3.2 Le chargement automatique

Règle (§13.3). « *Packages are often inter-dependent, and loading one may cause others to be automatically loaded. The colon operators described above will also cause automatic loading of the associated package. When packages with namespaces are loaded automatically they are not added to the search list.* »

Trois faits en trois phrases, et le troisième explique le concept 1 :

1. charger un paquet peut **en charger d'autres** ;
2. `::` **suffit à déclencher un chargement** — écrire `pkg::f()` charge `pkg` si nécessaire ;
3. ces chargements automatiques **n'apparaissent pas dans** `search()` — d'où `loadedNamespaces()` .

Énoncé. Un utilisateur charge deux paquets. Depuis, `filter(x, rep(1,3))` ne fait plus ce qu'il attendait, et un message a signalé un objet masqué. Comment diagnostiquer et corriger ?

Étape 1 — voir l'ordre. `search()` donne le chemin, du plus prioritaire au moins :

```
search()
```

Le rappel de la fiche 306 : **le premier élément est `.GlobalEnv`** , le dernier `package:base` , et « *other environments are inserted in the path using `attach` or `library`* » (*R Language Definition* §3.5.4). **Un paquet chargé récemment est inséré près du début** — il masque donc ceux d'avant.

Étape 2 — voir ce que contient chaque position. `ls(2)` , `ls(3)` ... — « *Ls (or `objects`) can be used to **examine the contents of any position on the search path*** » (§6.3.5, fiche 306). On repère ainsi **quelles positions définissent `filter`** .

Étape 3 — comprendre le message de masquage. R signale, au chargement, les objets qu'un paquet masque. Ce n'est **pas une erreur** : c'est le fonctionnement normal du chemin de recherche, **la première correspondance gagne**.

Étape 4 — la correction immédiate. Nommer explicitement le paquet voulu :

```
stats::filter(x, rep(1, 3))
```

« **The double-colon operator `::` selects definitions from a particular namespace.** » L'écriture **ne dépend plus de l'ordre du chemin** — c'est sa vertu principale.

Étape 5 — pourquoi les paquets, eux, ne cassent pas. Parce qu'« **objects cannot be masked by objects of the same name in the global environment or in other packages** » dès qu'il y a un espace de noms : le code interne de `stats` trouve **son `filter`** avant tout le reste. **Seul l'utilisateur est exposé**.

Étape 6 — vérifier ce qui est réellement chargé. Si un `pkg::f()` a fonctionné sans `library(pkg)` , c'est le chargement automatique : « **The colon operators ... will also cause automatic loading of the associated package** ». Et ce paquet **n'est pas dans `search()`** — il faut `loadedNamespaces()` pour le voir.

Étape 7 — la règle à adopter. Dans un script destiné à durer, **préfixer par `::` toute fonction dont le nom est courant** : `filter` , `select` , `t` , `c` , `data` . Cela documente l'origine et **immunise contre l'ordre de chargement**.

Étape 8 — ce qu'il ne faut pas faire. Recourir à `:::` pour contourner un problème : l'objet visé est **délibérément caché**, hors du contrat public. Le cours oriente vers `getAnywhere()` « *which searches multiple packages* » — pour **lire**, non pour **dépendre**.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« quels paquets ai-je ? »	<code>library()</code> sans argument
« charger un paquet »	<code>library(nom)</code>
« installer / mettre à jour »	<code>install.packages()</code> · <code>update.packages()</code>
« qu'est-ce qui est chargé ? »	<code>search()</code> — puis <code>loadedNamespaces()</code>
« ma fonction a changé de comportement »	masquage — voir <code>search()</code> et <code>ls(n)</code>
« lever une ambiguïté de nom »	<code>paquet::fonction</code>
« accéder à une fonction interne »	<code>:::</code> — ou plutôt <code>getAnywhere()</code>
« pourquoi ce paquet n'est pas dans <code>search()</code> ? »	chargement automatique d'un paquet à espace de noms
« pourquoi mon paquet ne casse pas si je redéfinit <code>t</code> ? »	l'espace de noms
« où trouver l'aide d'un paquet ? »	<code>help.start()</code> , section Reference
« d'où viennent les paquets ? »	CRAN, Bioconductor , et autres dépôts
« comment écrire un paquet ? »	hors de ce chapitre — <i>Writing R Extensions</i> (fiche 319)

Comment résoudre ce type d'exercice

Protocole « mettre en place son environnement de travail » — 4 étapes.

1. `library()` pour voir ce qui est installé.
2. `install.packages("nom")` si nécessaire — connexion requise.
3. `library(nom)` au début du script, **une fois**.
4. Vérifier avec `search()` — et `loadedNamespaces()` pour ce qui n'y figure pas.

Protocole « écrire du code robuste aux conflits » — 4 étapes.

1. Préfixer par `paquet::` toute fonction dont le nom est courant ou ambigu.

2. Ne **jamais** dépendre de l'ordre de chargement des paquets.
3. Éviter `:::` : ce qui est caché **n'est pas un contrat**.
4. Pour lire une définition interne, utiliser `getAnywhere()` — ou `getS3method()` pour une méthode S3 (fiche 311).

● Common mistakes

Erreur	Correction
Croire toutes les fonctions disponibles d'emblée	« <i>Only when a package is loaded</i> are its contents available »
Croire <code>search()</code> exhaustif	certain paquets sont chargés sans y figurer
S'appuyer sur l'ordre de chargement	utiliser <code>::</code>
Utiliser <code>:::</code> pour contourner	l'objet est délibérément caché → <code>getAnywhere()</code>
Croire <code>::</code> capable d'atteindre un objet interne	« <i>Only functions that are exported</i> ... can be retrieved in this way »
S'étonner qu'un paquet non chargé explicitement soit actif	interdépendances et chargement par <code>::</code>
Croire qu'un paquet casse si l'on redéfinit un de ses noms	l' espace de noms l'en protège
Confondre « installé » et « chargé »	<code>library()</code> liste les installés , <code>search()</code> les chargés et attachés
Se fier à la liste de la R FAQ	« <i>the collection ... changes very frequently</i> »
Chercher la création de paquets ici	c'est Writing R Extensions

Ultimate Review

Le principe. « ***All R functions and datasets are stored in packages. Only when a package is loaded are its contents available.*** » Deux raisons : **l'efficacité** (« *more memory ... longer to search* ») et **la protection des développeurs** (« *protected from name clashes with other code* »).

Les commandes. `library()` — les paquets **installés** · `library(nom)` — **charger** · `install.packages()` / `update.packages()` — installer et mettre à jour, « *available through the **Packages menu*** » · `search()`

— la **liste de recherche** · `loadedNamespaces()` — ceux « *loaded but not available on the search list* » · `help.start()` puis la section **Reference**.

Trois catégories. **Standard** (*base*) — « *considered part of the R source code ... automatically available in any R installation* » · **recommandés** — « *distributed with every binary distribution* » · **contribués** — « *thousands ... written by many different authors* », sur **CRAN**, **Bioconductor** et autres dépôts ; « *the collection ... changes very frequently* ».

Un espace de noms fait trois choses. (1) « *hide functions and data that are meant only for internal use* » ; (2) « *prevent functions from breaking when a user (or other package writer) picks a name that clashes* » ; (3) « *provide a way to refer to an object within a particular package* ».

L'exemple. « `t()` is the transpose function in R, but users might define their own function named `t`. Namespaces prevent the user's definition from taking precedence, and breaking every function that tries to transpose a matrix. »

Le chemin de recherche d'un paquet à espace de noms (*R Language Definition* §3.5.4) : **le paquet** → **ses imports** → `base` → l'environnement global → **le reste** ; « *objects cannot be masked by objects of the same name in the global environment or in other packages* ».

Deux opérateurs. `::` — « *selects definitions from a particular namespace* », « *only functions that are exported ... can be retrieved in this way* » · `:::` — « *acts like the double-colon operator but also allows access to hidden objects* », « *Users are more likely to use the `getAnywhere()` function, which searches multiple packages.* »

Le chargement automatique. « *Packages are often inter-dependent, and loading one may cause others to be automatically loaded. The colon operators ... will also cause automatic loading. When packages with namespaces are loaded automatically they are not added to the search list.* »

Pour aller plus loin. « *The process of developing packages is described in Section "Creating R packages" in Writing R Extensions* » — fiche 319.

Active Recall

« *All R functions and datasets are stored in packages. Only when a package is loaded are its contents available. This is done both for efficiency (the full list would take more memory and would take longer to search than a subset), and to aid package developers, who are protected from name clashes with other code.* » (§13)

Deux raisons de nature différente : l'une est **un coût** (mémoire et temps de recherche), l'autre est **une garantie de correction** — un paquet doit fonctionner quoi que l'utilisateur définisse par ailleurs.

C'est la seconde qui donne naissance aux **espaces de noms** (§13.3), et qui résout le problème de masquage rencontré au chapitre 6 (fiche 306).

*« To see which packages are currently loaded, use `search()` to display the search list. **Some packages may be loaded but not available on the search list ... these will be included in the list given by `loadedNamespaces()`** . »* (§13)

La raison est donnée au §13.3 : « *When packages with namespaces are loaded automatically they are not added to the search list.* »

Un paquet chargé par **dépendance** — ou par un simple `pkg::f()` — est **actif en mémoire** sans figurer dans `search()` . Se fier au seul `search()` donne donc une vue **incomplète** de ce qui tourne.

Catégorie	Ce qu'en dit le cours
Standard (base)	« <i>considered part of the R source code ... contain the basic functions that allow R to work ... should be automatically available in any R installation</i> » (§13.1)
Recommandés	« <i>Some (the recommended packages) are distributed with every binary distribution of R</i> » (§13.2)
Contribués	« <i>There are thousands ... written by many different authors</i> » — méthodes spécialisées, accès à des données ou du matériel, compléments de manuels

Le §1.3 avait donné le total des deux premières : « *about 25 packages supplied with R* ».

Les contribués viennent de **CRAN** et d'autres dépôts comme **Bioconductor** — « *the collection of available packages changes very frequently* ».

« *Packages have namespaces, which do three things* » (§13.3) :

1. « *they allow the package writer to **hide functions and data that are meant only for internal use*** » ;
2. « *they **prevent functions from breaking when a user (or other package writer) picks a name that clashes with one in the package*** » ;
3. « *they **provide a way to refer to an object within a particular package*** ».

Cacher, protéger, désigner. L'exemple du cours illustre la deuxième : « `t()` is the transpose function ... **Namespaces prevent the user's definition from taking precedence, and breaking every function that tries to transpose a matrix.** »

« *The double-colon operator `::` selects definitions from a particular namespace ... Only functions that are exported from the package can be retrieved in this way.* »

« *The triple-colon operator `:::` ... acts like the double-colon operator but also allows access to hidden objects. Users are more likely to use the `getAnywhere()` function, which searches multiple packages.* » (§13.3)

	Portée
<code>pkg::f</code>	ce qui est exporté
<code>pkg>:::f</code>	y compris ce qui est caché
<code>getAnywhere("f")</code>	cherche dans plusieurs paquets

Le cours oriente vers `getAnywhere()` parce qu'un objet caché relève du **premier rôle** de l'espace de noms — il est « *meant only for internal use* », donc hors du contrat public : il peut changer sans préavis.

« *Packages are often inter-dependent, and loading one may cause others to be automatically loaded. The colon operators described above will also cause automatic loading of the associated package. When packages with namespaces are loaded automatically they are not added to the search list.* » (§13.3)

Trois conséquences. (1) L'appel **fonctionne** : `::` charge le paquet si besoin. (2) Le paquet **n'apparaît pas dans `search()`**. (3) Ses fonctions **ne sont pas accessibles sans préfixe** — puisqu'il n'est pas sur le chemin.

C'est un comportement utile : on peut **dépendre ponctuellement** d'un paquet sans encombrer le chemin de recherche ni risquer de masquer quoi que ce soit.

Parce que le chemin de recherche **d'un paquet à espace de noms** n'est pas celui de l'utilisateur (*R Language Definition* §3.5.4) :

le paquet → ses imports → **base** → **.GlobalEnv** → le reste

« *The effect is that references to other objects in the same package will be resolved to the package, and objects cannot be masked by objects of the same name in the global environment or in other packages.* »

Le paquet passe avant tout. Le `t` de l'utilisateur, qui vit dans `.GlobalEnv`, arrive **après** `base` dans cette chaîne — il ne peut donc rien masquer côté paquet.

Mais l'utilisateur, lui, reste exposé : dans sa session, `t` désigne bien sa définition. D'où l'intérêt de `base::t`.

Flashcards

Question	Réponse
Où sont rangées toutes les fonctions de R ?	Dans des paquets
Quand leur contenu est-il disponible ?	Seulement une fois le paquet chargé
La première raison de ce découpage ?	L' efficacité — mémoire et temps de recherche
La seconde ?	Protéger les développeurs des collisions de noms
Voir les paquets installés ?	<code>library()</code> sans argument
Charger un paquet ?	<code>library(nom)</code>
Le paquet donné en exemple ?	<code>boot</code> — Davison & Hinkley (1997)
Installer et mettre à jour ?	<code>install.packages()</code> · <code>update.packages()</code>
Où trouver ces commandes en interface graphique ?	Le menu Packages
Voir ce qui est chargé et attaché ?	<code>search()</code>
Et ce qui est chargé sans y être ?	<code>loadedNamespaces()</code>
Voir les rubriques d'aide d'un paquet ?	<code>help.start()</code> , section Reference
Que sont les paquets standard ?	« part of the R source code »
Sont-ils toujours disponibles ?	Oui , dans toute installation
Que sont les paquets recommandés ?	Livrés avec toute distribution binaire
Combien de paquets contribués ?	Des milliers
Deux dépôts cités ?	CRAN et Bioconductor
La liste de la R FAQ est-elle à jour ?	La collection change très fréquemment
Combien de choses fait un espace de noms ?	Trois
La première ?	Cacher ce qui est à usage interne
La deuxième ?	Empêcher les collisions de casser le paquet

Question	Réponse
La troisième ?	Désigner un objet d'un paquet précis
L'exemple du cours ?	<code>t()</code> , la transposition
Que fait <code>::</code> ?	Il sélectionne dans un espace de noms
Sa limite ?	Seulement ce qui est exporté
Que fait <code>:::</code> ?	Comme <code>::</code> , plus les objets cachés
Que recommande le cours à sa place ?	<code>getAnywhere()</code>
Pourquoi ?	Il cherche dans plusieurs paquets
Charger un paquet peut-il en charger d'autres ?	Oui — interdépendances
<code>::</code> déclenche-t-il un chargement ?	Oui
Ces paquets entrent-ils dans <code>search()</code> ?	Non
L'ordre de recherche depuis un paquet ?	Paquet → imports → <code>base</code> → global → reste
Que garantit cet ordre ?	Qu'un objet ne peut pas être masqué
Où lire comment écrire un paquet ?	Writing R Extensions